Testbench file:

```
`timescale 1ns/1ps

module password_lock_tb;

reg clk;
reg reset;
reg [3:0] entered_password;

wire access_granted;
wire locked;

password_lock uut(
   .clk(clk),
   .reset(reset),
   .entered_password(entered_password),
   .access_granted(access_granted),
   .locked(locked)
);

always #5 clk = ~clk;

initial
begin

   $dumpfile("password_lock.vcd");
   $dumpvars(0,password_lock_tb);

   clk = 0;
   reset = 1;
   entered_password = 4'b0000;

   #10;
   reset = 0;

   // Wrong Attempt 1
   entered_password = 4'b1111;
   #10;
   $display("Attempt 1 -> Password=%b  Access=%b  Locked=%b",
         entered_password, access_granted, locked);

   // Wrong Attempt 2
   entered_password = 4'b0001;
   #10;
   $display("Attempt 2 -> Password=%b  Access=%b  Locked=%b",
         entered_password, access_granted, locked);
```

```
    // Wrong Attempt 3
    entered_password = 4'b0010;
    #10;
    $display("Attempt 3 -> Password=%b  Access=%b  Locked=%b",
            entered_password, access_granted, locked);

    // Correct password after lock
    entered_password = 4'b1011;
    #10;
    $display("Correct Password After Lock -> Password=%b  Access=%b  Locked=%b",
            entered_password, access_granted, locked);

    // Reset
    reset = 1;
    #10;
    reset = 0;

    // Correct password after reset
    entered_password = 4'b1011;
    #10;
    $display("After Reset -> Password=%b  Access=%b  Locked=%b",
            entered_password, access_granted, locked);

    $finish;

end

endmodule
```